

How the Chess Bot Thinks

A guide for chess players — no programming knowledge required

ClaudeChessV2 · bot2.py

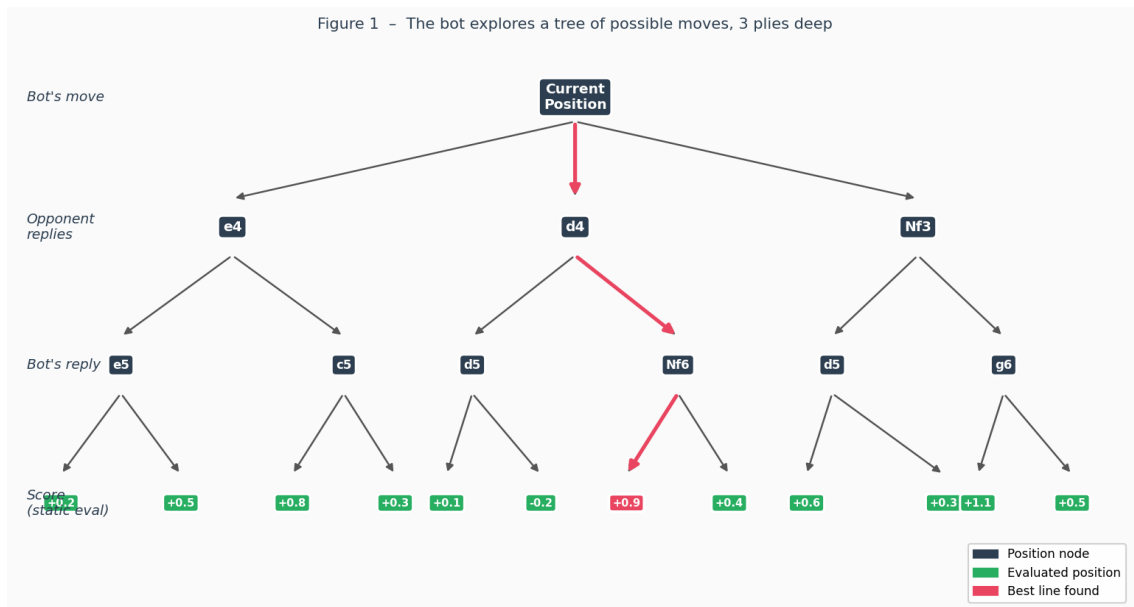
This document explains the ideas behind a computer chess engine. Every concept is described purely in chess terms — no code, no formulas. By the end you will understand exactly how the bot decides which move to play, why it sometimes blunders, and how each technique was added to make it stronger.

1. The Core Idea — Searching Ahead

When a strong human player considers a move, they think ahead: *"If I play e4, my opponent might reply e5 or c5 or d5 — and then if I play Nf3..."* A chess engine does exactly the same thing, but mechanically and at great speed.

The bot builds a **game tree**: a branching structure where every node is a position and every edge is a move. Starting from the current position it explores all of its candidate moves, then for each of those it explores all opponent replies, and so on.

The bot imagines it is **both players at once**. When it is its own turn it picks the move with the *highest* score; when it is the opponent's turn it assumes the opponent will pick the move with the *lowest* score (worst for the bot). This alternating logic is called **minimax**.



The red path shows the best line found: d4 → Nf6 → (score +0.9). All leaf nodes (green) have been scored by the evaluation function.

A key term is **ply**: one half-move (one player's turn). Searching 6 plies means looking 3 full moves ahead. The current bot routinely searches **8–9 plies** within its half-second time budget.

2. Alpha-Beta Pruning — Ignoring Dead Ends

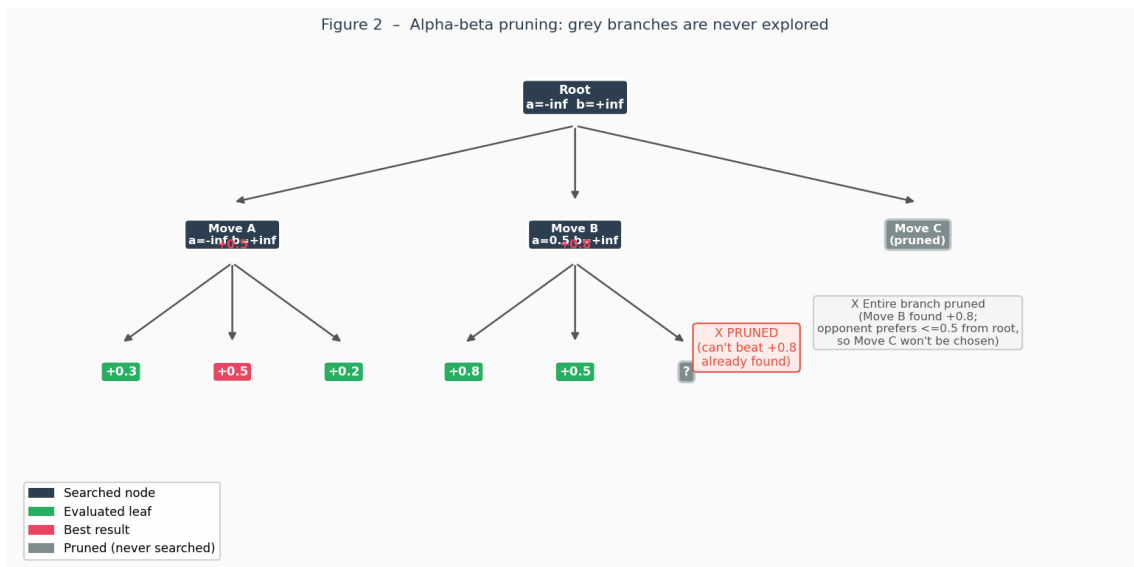
A naive minimax search is too slow: with an average of ~30 legal moves per position, searching 8 plies would require examining $30^8 = 656$ billion positions. That is impossible in half a second.

The solution is **alpha-beta pruning**. The idea is simple: if you have already found a move that scores +0.8, and you start examining another move that the *opponent* can already refute to +0.5, you can stop looking — that line can never be better than the one you already have. You **prune** the rest of that branch.

Two values track what is possible:

- **Alpha (a)** — the best score the bot can *guarantee* so far. It will never accept less than this.
- **Beta (b)** — the best score the *opponent* can guarantee. Upper bound on what the bot can hope for.

Whenever $\alpha \geq \beta$, the bot stops — the opponent would never let this position arise. In practice alpha-beta cuts the search to roughly **the square root of total nodes**, doubling the effective search depth for the same computation time.



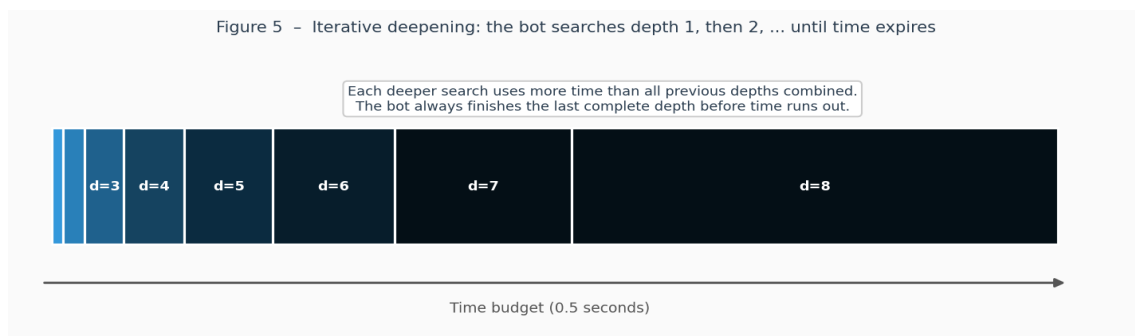
Grey nodes are never evaluated. The entire Move C branch is skipped because Move B already proved at least +0.8 for White.

3. Iterative Deepening — Using Every Millisecond

The bot has a fixed time budget — 0.5 seconds per move. It cannot know in advance how deep it can search within that budget, because different positions have different numbers of legal moves.

The solution is **iterative deepening**: the bot first searches to depth 1 (one ply), then restarts and searches to depth 2, then depth 3, and so on. Each complete depth replaces the previous best move. When time runs out, the bot plays the best move found at the *last fully completed depth*.

This sounds wasteful — why redo all previous work? The key insight is that **deeper searches take exponentially longer**. Depth 8 takes more time than depths 1 through 7 *combined*. So the repeated shallower searches are a tiny overhead.



Almost all the time budget is spent on the deepest completed depth.

There is another benefit: the best move found at depth 7 is used to **order moves** in the depth-8 search (it is tried first). Because alpha-beta pruning works best when strong moves come first, this makes the whole search significantly more efficient.

4. Move Ordering — Best First

Alpha-beta pruning saves the most work when **good moves are tried first**. If the best move is examined first, all other moves get pruned quickly. The bot uses several techniques to put strong moves at the top of its list:

MVV-LVA (Most Valuable Victim, Least Valuable Attacker)

Captures are always tried before quiet moves. Among captures, the bot prefers **taking the most valuable piece with the least valuable attacker**: Pawn×Queen scores higher than Queen×Pawn. This simple rule orders nearly all captures correctly.

Transposition Table (Memory of Past Searches)

The bot stores the best move found for each position in a **lookup table** (up to 100,000 entries). At depth 8, many positions were already seen at depth 7. The stored best move from depth 7 is tried first at depth 8 — often immediately triggering a cutoff and saving the rest of the search.

Killer Moves

When a quiet move (a non-capture) causes a cutoff, it is stored as a **killer move** for that depth level. In other branches at the same depth, the killer is tried before other quiet moves. The idea is that a strong move at one node is often strong in sibling positions too — even if the boards look different.

History Heuristic

Every time a quiet move causes a cutoff anywhere in the tree, its score in a **history table** is increased. Moves that have been useful many times across the whole search get tried earlier. This is a statistical measure of a move's general aggressiveness.

5. Quiescence Search — Resolving Captures

Even at depth 8, the bot must eventually stop and **score a position with a static evaluation function**. But what if the position at the search horizon is mid-exchange — a queen just captured a pawn and the recapture hasn't happened yet?

The Horizon Effect: without special treatment, the bot might play Qxf7 at depth 8, score the position as +1.4 (pawn gained, check bonus), and think it has found a brilliant move — not seeing that Kxf7 on the very next ply wins the queen back and leaves the bot down material.

The solution is **quiescence search**: when the main search reaches depth 0, the bot does *not* immediately evaluate the position. Instead it keeps searching, but only captures, until a **quiet position** is reached (no more worthwhile captures available). Only then is the position scored.

The key mechanism is the **stand-pat option**: at each step the bot checks whether simply *not capturing* is already good enough. If the static evaluation is already above the cutoff threshold, it returns immediately without searching further.

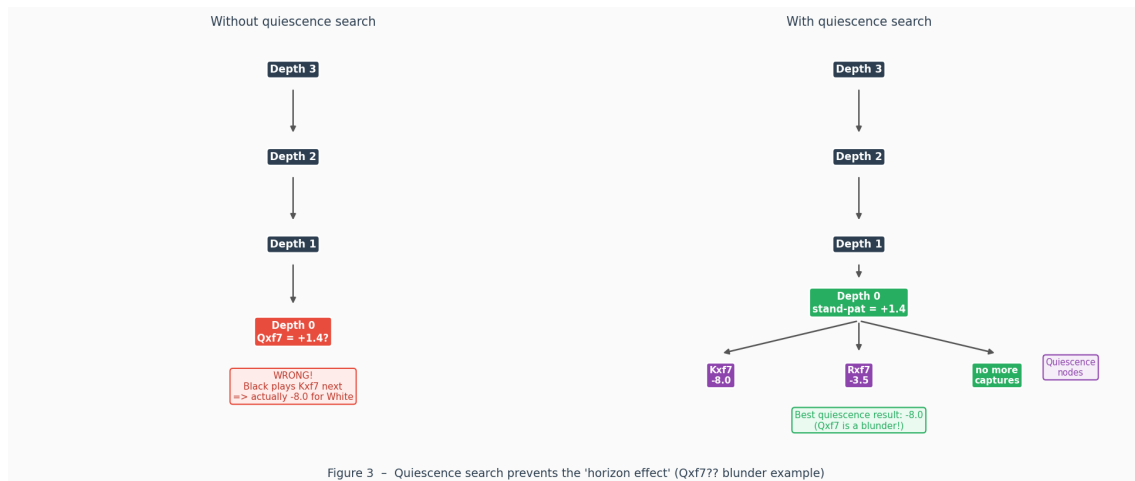


Figure 3 - Quiescence search prevents the 'horizon effect' (Qxf7?? blunder example)

Without quiescence (left), Qxf7 looks like +1.4. With quiescence (right), Kxf7 is found immediately and the move is correctly rejected.

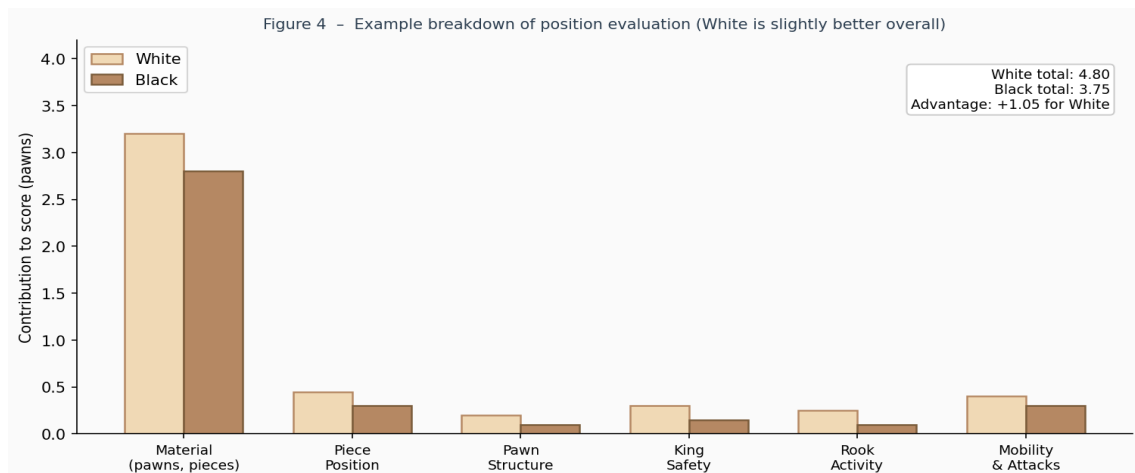
To keep quiescence fast, only **winning or equal captures** are searched. A capture where the attacker is worth three times more than the victim (e.g. Queen takes Pawn) is skipped — the bot assumes it is losing and uses the stand-pat score instead.

Result: Adding quiescence search raised the bot's win rate against the previous version from 90% to **100%** (10 wins, 0 draws, 0 losses). Tactical blunders dropped from 34 to 9 per game.

6. Evaluating a Position

When the search reaches a quiet position and must return a score, it calls the **evaluation function**. This is the bot's judgment about who stands better, expressed in **centipawns** (100 centipawns = one pawn).

The evaluation adds up contributions from several factors:



Example evaluation breakdown. White leads in material and piece activity, giving an overall advantage of roughly +0.8 pawns.

Material

The most important factor. Pawn=1, Knight=3, Bishop=3, Rook=5, Queen=9. Being up a pawn is +100 centipawns; being down a piece is -300.

Piece-Square Tables

The same piece is worth more on some squares than others. Knights on the edge get a penalty; knights in the centre get a bonus. A castled king on g1 gets a bonus; an early king in the centre gets a penalty. These bonuses and penalties are stored in lookup tables — one per piece type.

Pawn Structure

- **Doubled pawns** — two pawns on the same file; penalty. They block each other and can't protect one another.
- **Isolated pawns** — a pawn with no friendly pawns on adjacent files; penalty. Easily attacked, hard to defend.
- **Passed pawns** — a pawn with no enemy pawns in front of it on adjacent files; bonus scaling with rank. A passed pawn on the 7th rank (one step from promotion) is worth +100 centipawns extra.

Rook Activity

- **Open file** (no pawns at all on the column): +50 cp bonus.

- **Semi-open file** (own pawns absent, enemy pawns present): +25 cp.

King Safety

In the middlegame a king is safest behind its castled pawn shelter. The bot gives a bonus for each pawn directly in front of the castled king, and a smaller bonus for pawns two squares ahead. A broken pawn shield is penalised.

Bishop Pair Bonus

A side that keeps both bishops while the opponent has lost one gets a bonus. Two bishops together cover all squares and dominate open positions.

Mobility and Attack Counting

For each piece, the bot counts how many squares it attacks. Pieces with more scope score higher. Attacking squares near the enemy king is especially valuable.

7. Advanced Search Techniques

Beyond the core minimax and alpha-beta, the bot uses two more powerful techniques to search deeper within the time budget:

Null Move Pruning

Imagine a position is so good that even if you **gave your opponent an extra free move**, you would still be winning. In that case, your actual best move is certainly also winning — you can prune the entire branch.

The bot implements this by temporarily passing its turn, letting the opponent move twice in a row, and searching that position at reduced depth. If the result still beats the cutoff threshold, the branch is pruned without further work.

***Important guard:** null move pruning is disabled in the endgame when few pieces remain. In those positions passing can actually help (zugzwang — the side to move would prefer to do nothing). The bot counts non-pawn pieces and disables the null move if fewer than 5 remain.*

Late Move Reductions (LMR)

With good move ordering, the first few moves at any node are almost certainly the strongest. Moves tried late in the list (move 6 and beyond) are probably weak. The bot searches these **late quiet moves at reduced depth** (one ply less) as a quick check. If the reduced search says the move is bad, it is skipped. Only if it surprisingly looks good is it re-searched at full depth.

LMR is only applied to quiet moves (non-captures) and only when the bot is not in check. The combination of LMR and null move pruning is what allows the bot to reach **depth 8–9** in half a second — without them it would only reach depth 5–6.

8. Legal Move Generation — Pin Detection

At every node in the search tree, the bot must generate all **legal** moves. A naive approach would try each candidate move, make it on the board, check whether the king ends up in check, and undo the move if illegal. This is correct but slow.

The bot uses a smarter approach: before trying any moves, it casts **rays from its own king** in all 8 directions. If a ray hits a friendly piece and continues to an enemy slider (bishop, rook, or queen) before hitting anything else, that friendly piece is **absolutely pinned** — it cannot move without exposing the king to check.

***Chess example:** your bishop is on d4, your king is on a1, and the enemy queen is on g7. The bishop is on the a1-h8 diagonal between king and queen — it is pinned and cannot move off that diagonal. The bot detects this in advance and skips the legality check for all moves the bishop could make off the diagonal.*

King moves and en passant captures are always verified separately. All other non-pinned pieces skip the expensive legality check entirely.

***Result:** This single change increased average search depth from 7.6 to 8.8 plies (+1.2 ply, ~17% deeper search).*

9. Putting It All Together

Here is the full sequence the bot follows when it is time to move:

Step	What happens
Start the clock	0.5 seconds on the clock.
Depth 1 search	Search all moves 1 ply deep, rank them by score. Pick tentative best move.
Depth 2 search	Restart. Try the depth-1 best move first (TT ordering). Apply alpha-beta to prune bad branches. Update best move.
Depth 3, 4, ... N	Repeat, going one ply deeper each time. At each node: check for null-move pruning first. For each move: check for killer move pruning. For each move: check for MVV-LVA pruning.
Time runs out	Play the best move from the last fully completed depth. Any incomplete depth is discarded.

Performance Summary

Technique	What it does	Impact
Minimax + Alpha-Beta	Explore game tree, prune dead ends	Core search — enables everything else
Iterative Deepening	Search deeper until time runs out	Best move always available; improves ordering
Move Ordering (TT + killers + MVV-LVA)	Try best moves first	~2x more effective pruning; +1-2 ply effective depth
Null Move Pruning	Skip turn to test if position is overwhelmingly good	Prunes 20-40% of branches in middlegame
Late Move Reductions (LMR)	Search weak late moves at reduced depth	Saves ~30% of nodes; allows depth 8-9 in 0.5 s
Pin Detection	Skip legality checks for non-pinned pieces	+1.2 ply average depth increase
Precomputed Attack Tables	Instant lookup instead of move generation in eval	~10x faster evaluation function
Quiescence Search	Resolve captures before scoring leaf nodes	100% win rate; blunders 34 -> 9 per game

The bot currently searches around 3,000–5,000 positions per move, reaching a depth of 8–9 plies in its 0.5-second budget. It wins consistently against an earlier version and beats a casual human player.